FakeCloud vs. LocalStack

AWS development on your laptop



Gavril Ștefan-Dorian

29 October 2026

From Amazon’s infrastructure to AWS

AMAZON

More customers
More requests
More stored data

→

Reusable infrastructure

Computing and storage become capabilities other developers can request through APIs.

2006: S3 and EC2 expose storage and computing through web-service interfaces.

Unexpected AWS bills

Idle resources

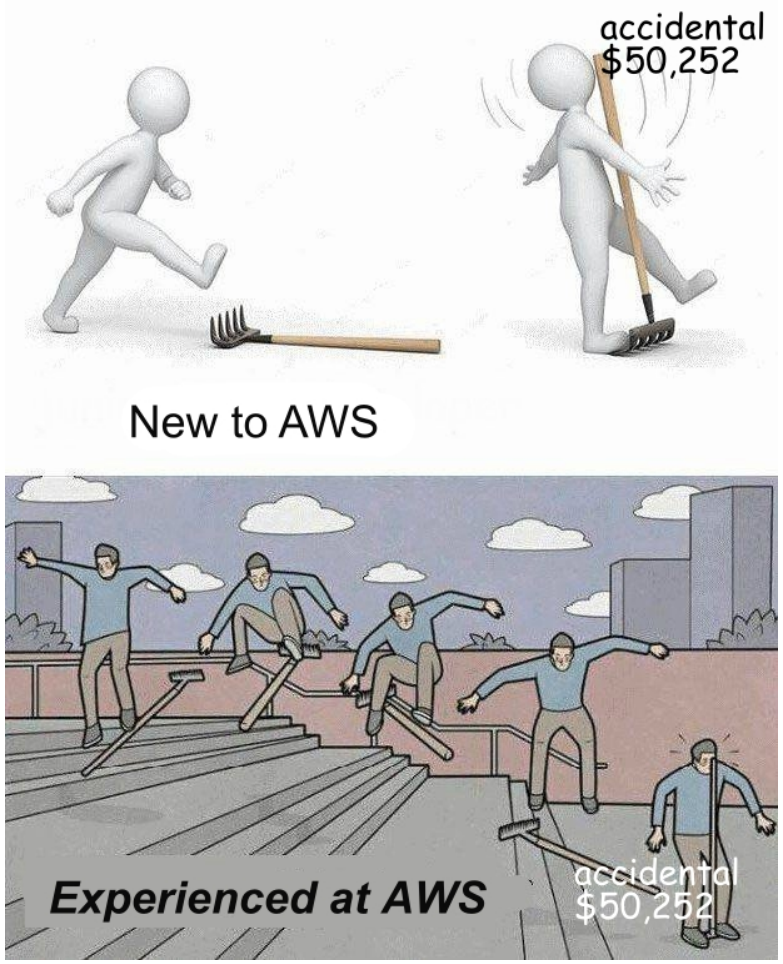
A running EC2 instance can cost money even when nobody uses the app.

Resources left behind

Stopping an instance does not remove its billable EBS storage.

Usage adds up

S3 storage, requests and data transfers have separate cost components.



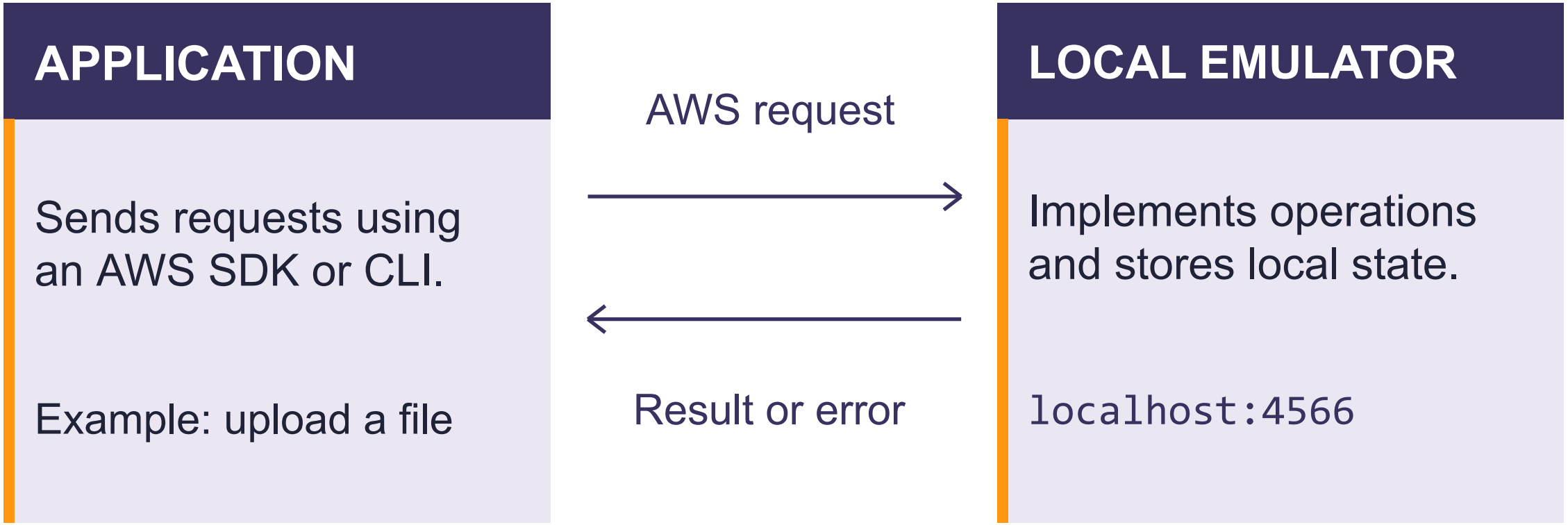
What slows down cloud development?

Waiting for feedback	Change a handler, deploy it, trigger it and fetch its logs before trying the next fix.
Access failures	A wrong permission or network rule can block a request even when the code is correct.
Shared resources	A teammate clears the shared queue while you are reproducing a failed job.
Leftover data	Old messages or records change today’s result, even when you repeat the same input.

What can we move onto the laptop?

Daily development	Run the changed code against local queues or storage and inspect the result nearby.
Debugging	Recreate the same input and failure in your own environment.
Team work	Use separate resources and data instead of changing a teammate’s experiment.
Automated checks	Start local services, create test data, run checks and remove the temporary environment.

How does a local AWS emulator work?



LocalStack began with an offline-development need

ORIGINS

Started at Atlassian in 2016.

Initially covered eight AWS services.

Exposed AWS-style APIs locally.

ORIGINAL NEED

Continue cloud-application development without calling remote AWS.

Use local services for development and testing, including while travelling offline.

From an offline tool to a development platform

More services	Combine local storage, queues, functions and databases in the same environment.
Infrastructure as code	Use Terraform or CDK to create the resources defined by your project.
Daily workflow	Run checks in CI, browse resources and debug functions with familiar tools.

Teams began depending on LocalStack to prepare and run their development environments.

What changed in March 2026?

BEFORE	23 MARCH 2026	FREE OPTION
Community image	**One consolidated image**	**Hobby plan**
Could start without a LocalStack account.	Startup now requires a LocalStack activation token.	Still available for eligible non-commercial use.

Existing CI jobs need a token. Account and plan now determine access.

Reported example: game-ci pinned an older image after its CI startup failed.

Why did the account requirement bother developers?

Extra setup	Create an account and provide a product token before starting the current image.
CI configuration	Store and supply an activation secret wherever automated checks start LocalStack.
Use restrictions	Check whether Hobby applies or the project needs a commercial plan.
Usage reporting	Activation is recorded. Optional usage events have a separate opt-out setting.

Why look beyond LocalStack?

One local dependency	A project using only a database or queue can run that dependency on its own.
Test integration	Replace AWS interactions within tests, or use a local HTTP server.
Local Lambda code	AWS SAM CLI runs functions and local API routes. Other service dependencies need separate setup.
Access and licence	Source access, commercial use and account requirements affect the choice.

Alternatives to LocalStack

TOOL	WHEN IT HELPS	MAIN LIMIT
Moto	AWS interactions in Python tests; HTTP server mode	Coverage depends on the operation.
DynamoDB Local	AWS-provided local DynamoDB database	DynamoDB only; no queues or functions.
ElasticMQ	Standalone SQS-compatible message queue	Queues only; no other AWS services.
S3Mock	S3 API with Docker and Java test integrations	A subset of S3; no wider AWS environment.

Why was FakeCloud created?

CI failures	Lucas Vieira reports broken builds after LocalStack introduced account activation.
Temporary workaround	Keep an older LocalStack release. The concern: how long can the project rely on it?
A new emulator	Started on 4 April 2026 to keep AWS-style integration testing free and open source.

Example: send a queue message and read back the same message.

What can you use for free?

FEATURE	FAKECLOUD	LOCALSTACK
Free use	AGPL-3.0; no paid tier	Hobby: non-commercial use
Service catalogue	100+ services advertised	Hobby: 30+; Base: 55+; Ultimate: 110+
Common AWS services	S3, SQS, Lambda, DynamoDB	Available in Hobby
User login	Cognito signup/login APIs	Base or Ultimate
Keep data on restart	Save local data to disk	Base or Ultimate

Breadth, depth and integration

Breadth	How many services are covered? Storage, queues, functions, databases and more.
Depth	How fully does each service behave like AWS? Older file versions and error handling are examples.
Integration	Do services work together? An S3 upload can notify a queue or trigger a Lambda function.

FakeCloud calls itself depth-first and LocalStack breadth-first.

What runs on your machine?

FAKECLOUD

One compiled Rust program.

Can run directly or inside a Docker container.

Service modules handle the AWS operations.

LOCALSTACK

A Python-based main runtime.

Runs inside the main LocalStack container.

Requests go to service implementations.

Running Lambda code adds Docker runtime containers in both tools.

Starting the emulators

FAKECLOUD

```
docker run --rm -it \
  -p 127.0.0.1:4566:4566 \
  ghcr.io/faiscadev/fakecloud
```

Docker running; no product account or token needed.

LOCALSTACK

```
docker run --rm -it \
  -p 127.0.0.1:4566:4566 \
  -e LOCALSTACK_AUTH_TOKEN \
  localstack/localstack
```

Set LOCALSTACK_AUTH_TOKEN in the shell before running this command.

Bash example for SQS. Start one emulator at a time on port 4566.

Create a queue with the AWS CLI

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1
unset AWS_SESSION_TOKEN

endpoint=http://localhost:4566
aws --endpoint-url "$endpoint" sqs \
  create-queue --queue-name wade-jobs
aws --endpoint-url "$endpoint" sqs \
  list-queues
```

Bash / AWS CLI · same commands for either emulator

Local settings

Dummy credentials and a region.

Create

Create the queue named wade-jobs.

Read back

List queues and look for wade-jobs.

Send and receive a message in Python

```
import boto3
endpoint = "http://localhost:4566"
sqs = boto3.client("sqs",
    endpoint_url=endpoint)
url = sqs.get_queue_url(
    QueueName="wade-jobs")["QueueUrl"]
sqs.send_message(QueueUrl=url,
    MessageBody="hello")
reply = sqs.receive_message(
    QueueUrl=url, WaitTimeSeconds=2)
for msg in reply.get("Messages", []):
    print(msg["Body"])
```

Python / Boto3 · either emulator

Connect

Either emulator at localhost:4566.

Send

Place the message hello in wade-jobs.

Receive

Read messages and print their contents.

Do errors behave like AWS?

```
HTTP/1.1 404 Not Found
Content-Type: application/xml

<Error>
  <Code>NoSuchKey</Code>
  <Message>
    The specified key does not exist.
  </Message>
</Error>
```

Expected S3 error · illustrative response

Request

Read a file that is not in an existing S3 bucket.

SDK error

The SDK recognises the code NoSuchKey.

App behaviour

Show “file not found” to the user.

Did the Lambda function run?

FAKECLOUD

```
GET
/_fakecloud/lambda/invocations
```

Recorded function calls with input and output.

Inspect the result and errors of an invocation.

LOCALSTACK

```
Log group:
/aws/lambda/<name>
```

Read function logs through CloudWatch Logs.

Look for invocation entries, console output and exceptions.

Check whether the function was invoked before debugging its code.

How do you start with an empty queue?

Empty one queue	PurgeQueue removes the messages while keeping the queue and its configuration.
Recreate a queue	DeleteQueue removes the queue. Create it again with the settings needed for the next run.
FakeCloud reset	POST /_fakecloud/reset/sqs clears all local SQS state, including queues and messages.

PurgeQueue and DeleteQueue work with both emulators.

Keeping data after a restart

FAKECLOUD · COMMAND

```
fakecloud \
  --storage-mode persistent \
  --data-path ./local-data
```

Save to ./local-data and reuse that directory after a restart.

LOCALSTACK · SETTING

```
PERSISTENCE=1
```

Save snapshots in a persistent, mounted data directory.

Requires Base or Ultimate.

Docker: keep the data directory outside the container.

Switching an existing project

WORKFLOW	FAKECLOUD	LOCALSTACK
AWS client	AWS SDKs and CLI; local endpoint	AWS SDKs and CLI; local endpoint
Resource tools	e.g. Terraform / AWS APIs	e.g. Terraform / CDK
Function edits	Lambda API updates	API updates; hot reloading
Tool-specific APIs	/_fakecloud/...	/_localstack/...

Can you test access restrictions?

FAKECLOUD

```
--iam strict
```

Policy enforcement is off by default.

test/test bypasses checks; use a restricted user.

LOCALSTACK

```
ENFORCE_IAM=1
```

Policy enforcement is off by default.

Requires Base or Ultimate and a suitable test identity.

Example: a read-only user should read a file but be refused permission to delete it.

Demo: graduate CSV processing

S3 INPUT	→	S3 EVENT	→	LAMBDA	→	S3 OUTPUT
Synthetic CSV		ObjectCreated		Validate + aggregate		JSON report

Same application

Two local emulators

Upload and inspect the report

Disconnect notification → no output

Reconnect and upload again → report

Open the live application: 127.0.0.1:8000

What still needs real AWS?

Email / SMS	FakeCloud records SNS email/SMS locally; messages are not actually delivered.
Deployment	Check the deployed roles, permissions and network paths in real AWS.
Event timing	An upload can finish before processing does; notifications may be delayed or repeated.
Scale / billing	Measure production capacity, latency and AWS charges in the real environment.

Where each tool has an advantage

CRITERION	FAKECLOUD	LOCALSTACK
Free local features	Cognito; persistence; IAM enforcement	Base/Ultimate for these features
Developer tools	Binary + inspection tools	Hot reload, CDK, support tiers
Track record	Started April 2026	Started 2016

FakeCloud offers more of these features without fees; LocalStack brings established tooling and support.

Thank you!

Selected sources

- **FakeCloud documentation**
  Implementation and service coverage
- **LocalStack documentation**
  Services and developer tools
- **FakeCloud vs. LocalStack**
  Vendor comparison
- **FAUN: FakeCloud vs. LocalStack**
  External review and comparison
- **LocalStack: March 2026 release**
  Account and activation changes

